

roboflow

Deepika

Premium Trial • 1 Month

Home

Projects

Workflows

Monitoring

Deployments

Explore

Settings

Help & Docs

Notifications

Deepika Grow...

0 credits used

Resets on February 28

Purchase Plan

Deepika

Search projects

Sort: Date Edited

17

Instance Segmentation

mine_detect 2

Edited 3 months ago

Private • 127 Images • 1 Models

Object Detection

mine_detect

Edited 3 months ago

Private • 127 Images • 2 Models

4

Invite Team

+ New Folder

+ New Project

0

roboflow

Let's create your project.

Deepika > Private Trial demo

Project Name

Annotation Group

Visibility

demo

object

Private

Public

Project Type

Object Detection

Bounding Boxes

Counts

Tracking

Identify objects and their positions with bounding boxes.

Classification

Image Labels

Filtering

Content Moderation

Assign labels to the entire image.

Single Label

Multi-Label

Instance Segmentation

Polygons

Measuring

Odd Shapes

Detect multiple objects and their actual shape.

Keypoint Detection

Skeleton Structure

Pose Estimation

Identify keypoints ("skeletons") on subjects.

Multimodal

Prompts

Visual Question Answering

Captions

Describe images using text pairs.

Show More



Cancel

Continue with Private

0

 Upload

Batch Name:

Uploaded on 02/10/26 at 4:53 pm

Tags:

Search or add tags for images...

☐ Create batch instantly

↑

Drag and drop file(s) to upload, or:

Select File(s)

Select Folder

Supported Formats

 Images

.jpg, .png, .bmp, .webp, .avif

in 26 formats

 Annotations

 Videos

.mov, .mp4

 PDFs

.pdf

Max size of 20MB and 10,000 x 10,000 pixels.

Need images to get started?



Upload data from your phone



Scan the QR code to upload images and videos from your phone directly to your project.



Search on Roboflow Universe

Search across 600k datasets and 400 million images in the world's largest platform for computer vision data.





Bulk Upload Images

Upload 1k+ images to Roboflow projects using Python SDK, REST API, and CLI.

[</> Learn More](#)

 Upload

Batch Name:

Folder: IMAGE

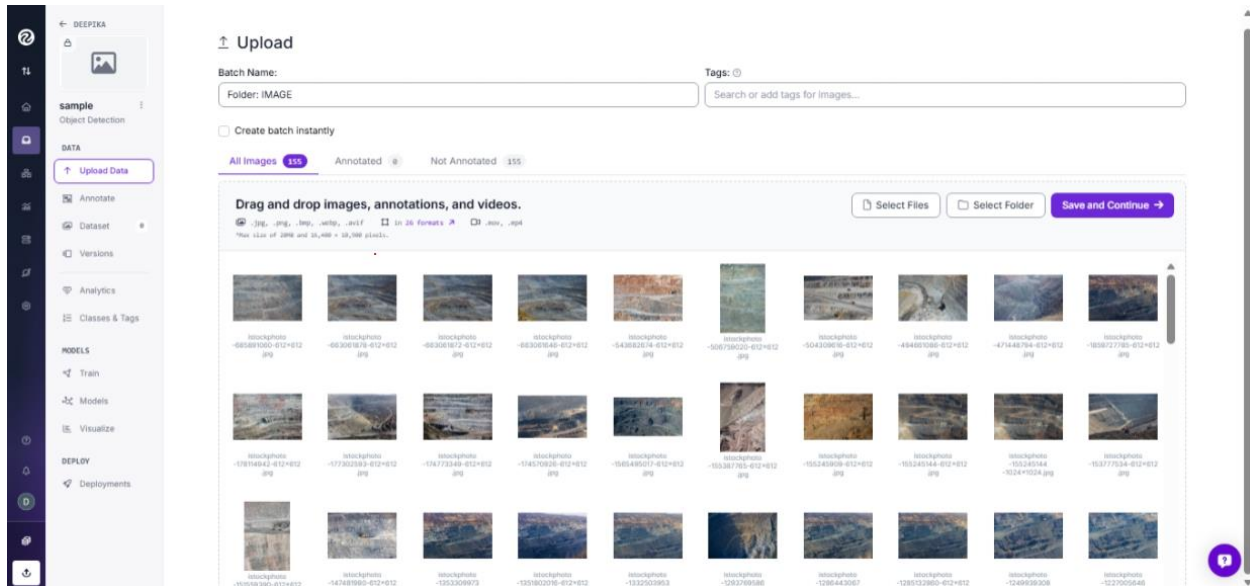
Tags: @

Search or add tags for images...

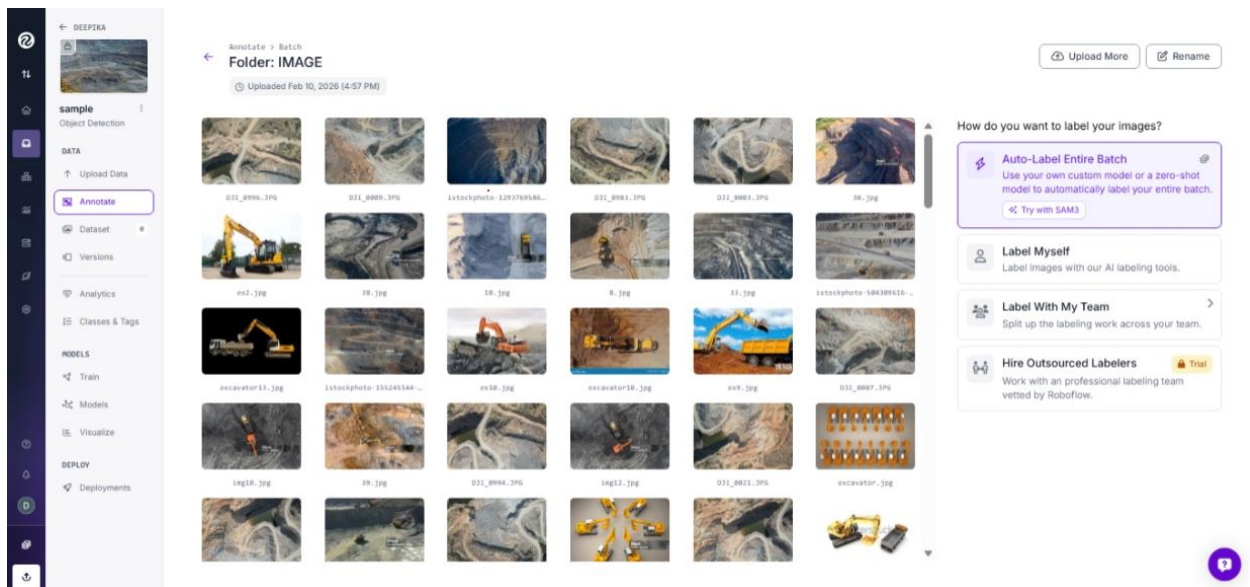
☐ Create batch instantly

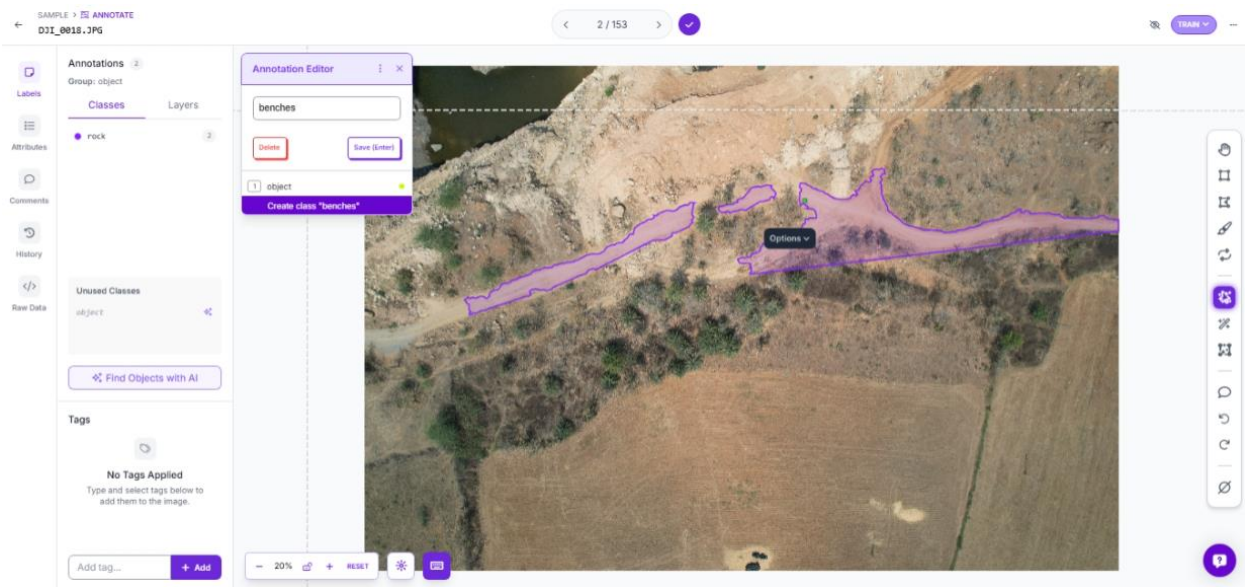
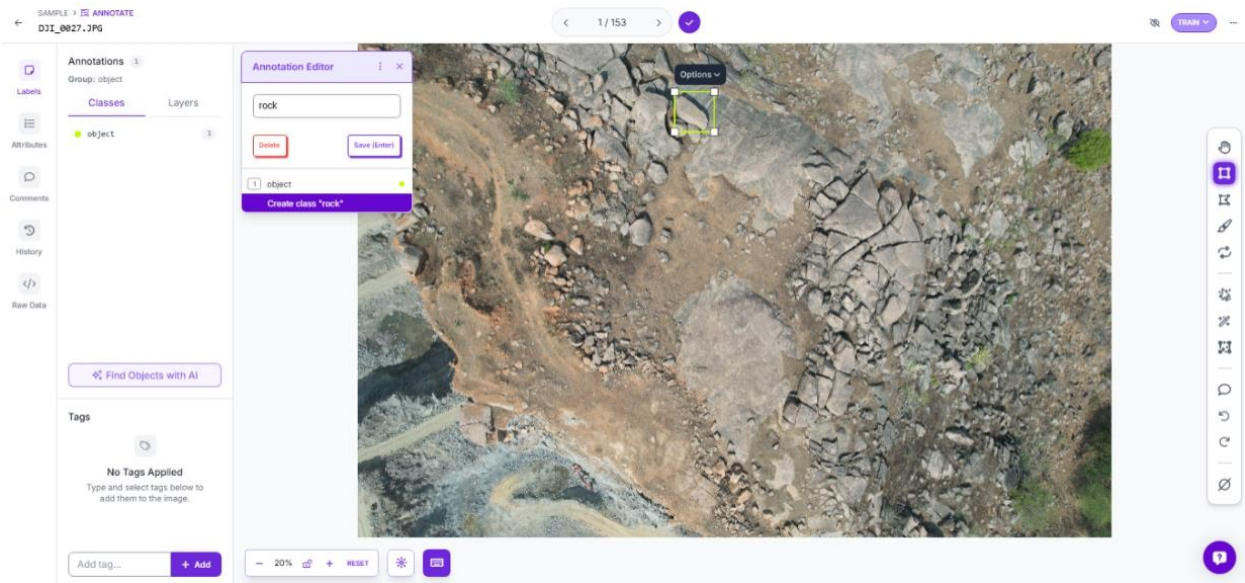
Processing files...

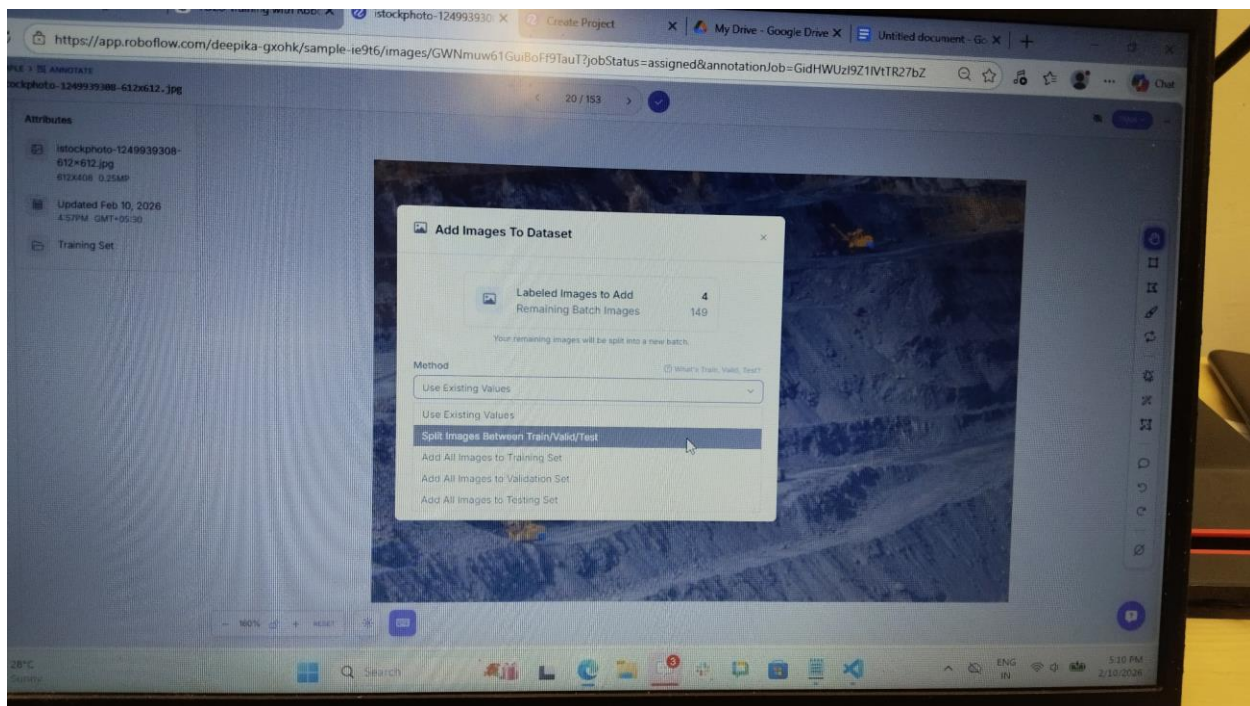
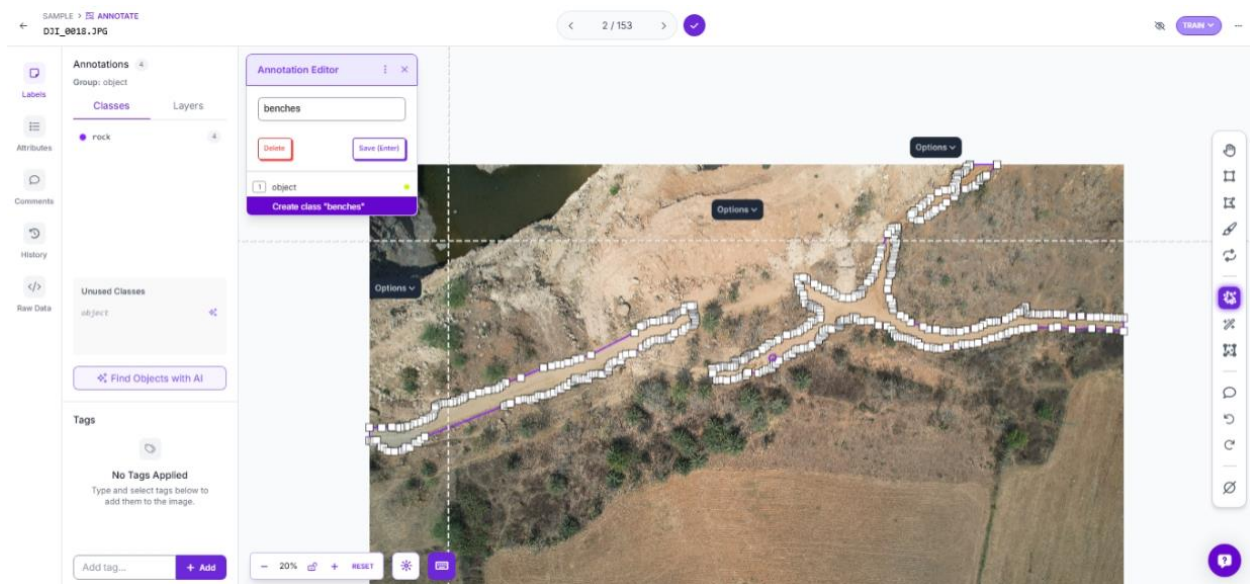
2PM68L-033Z-89999_3PG

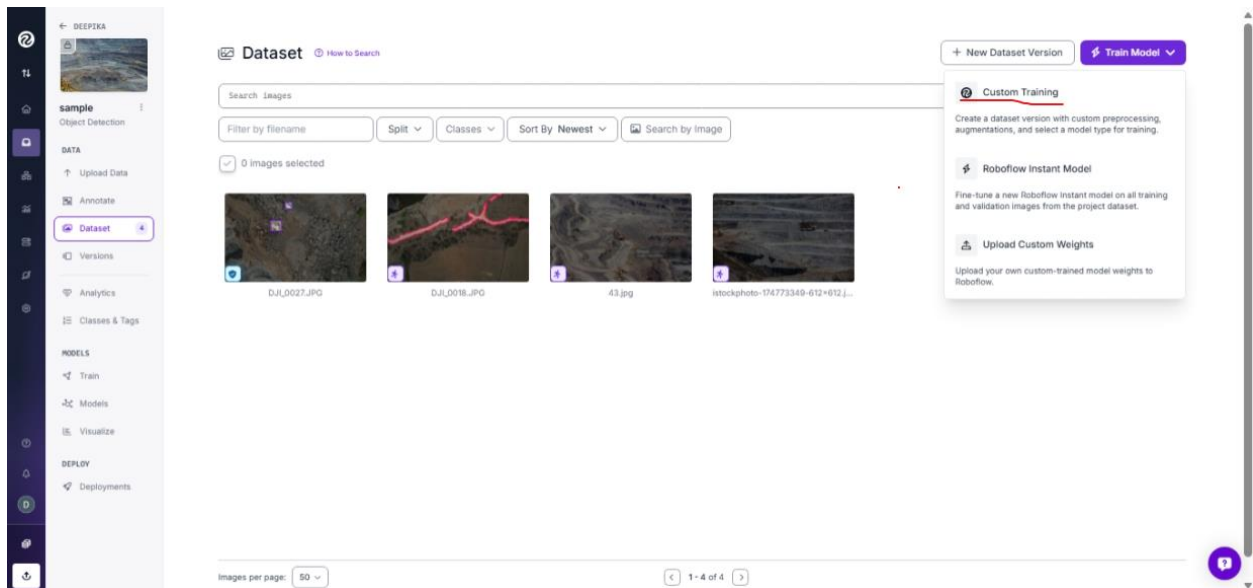
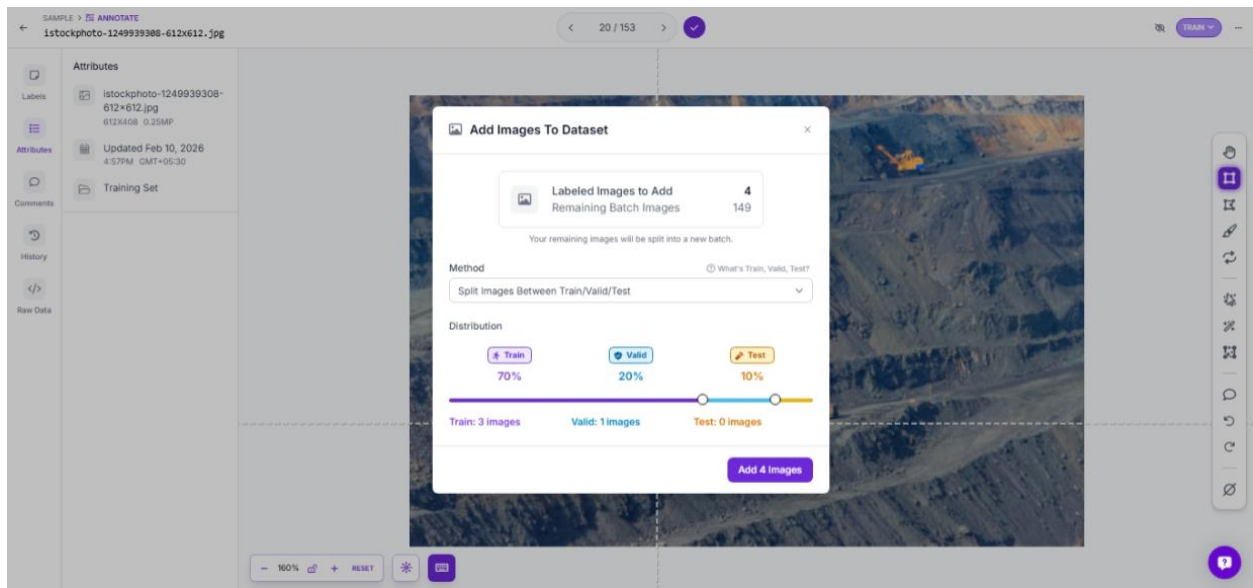


Click save and continue









DEEPIKA

sample

Object Detection

DATA

Upload Data

Annotate

Dataset 4

Versions

Analytics

Classes & Tags

MODELS

Train

Models

Visualize

DEPLOY

Deployments

Versions

Versions

Roboflow Instant 1 [Eval]

v1 4

Create New Version

Uses Credits

Prepare your images and data for training by compiling them into a version. Experiment with different configurations to achieve better training results.

Version Name:

demo

Source Images

Images: 4

Classes: 3

Unannotated: 0

Edit

Train/Test Split

Training Set: 3 images

Validation Set: 1 images

Testing Set: images

Edit

3 Preprocessing

Decrease training time and increase performance by applying image transformations to all images in this dataset.

Auto-Orient

Edit

x

Resize

Stretch to 512x512

Edit

x

DEEPIKA

sample

Object Detection

DATA

Upload Data

Annotate

Dataset 4

Versions

Analytics

Classes & Tags

MODELS

Train

Models

Visualize

DEPLOY

Deployments

Versions

Versions

Roboflow Instant 1 [Eval]

v1 4

Unannotated: 0

Train/Test Split

Training Set: 3 images

Validation Set: 1 images

Testing Set: images

Edit

Preprocessing

Auto-Orient: Applied

Resize: Stretch to 512x512

Edit

Augmentation

Turned Off

Edit

5 Create

Review your selections then click "Create" to create a moment-in-time snapshot of your dataset with the applied preprocessing steps.

Maximum Version Size: 4

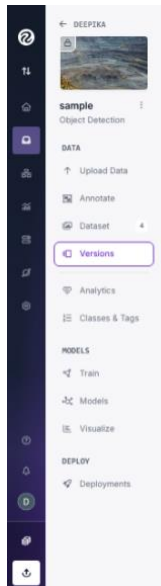
See how this is calculated

Version Notes:

Add any version notes here...

Back

Create



Versions

+ Create New Version

demo

Generated on Feb 10, 2026 by [deepika@velastra.co](#)

Download Dataset Edit

demo

v2 4 512x512 Stretch to [deepika@velastra.co](#)

Roboflow Instant 1 [Eval]

v1 4 [deepika@velastra.co](#)

Generated on Feb 10, 2026 by [deepika@velastra.co](#)

Download Dataset Edit

This version doesn't have a model.

Train an optimized, state of the art model with Roboflow or upload a custom trained model to use features like Label Assist and Model Evaluation and deployment options like our auto-scaling API and edge device support.

Train Model

Uses Roboflow Credits

[View usage](#)

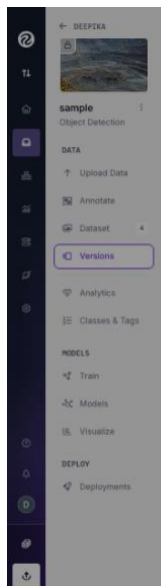
[How to Upload Custom Weights](#)

4 Total Images

[View All Images](#)



Dataset Split



Versions

+ Create New Version

Roboflow Instant 1 [Eval]

Generated on Feb 10, 2026 by [deepika@velastra.co](#)

Download Dataset Edit

Roboflow Instant 1 [Eval]

v1 4 [deepika@velastra.co](#)

Generated on Feb 10, 2026 by [deepika@velastra.co](#)

Download Dataset Edit

Download

Image and Annotation Format

YOLOv8

TXT annotations and YAML config used with YOLOv8.

Download Options

☒ Download zip to computer
Downloads all images, annotations, and classes.

☐ Show download code
Custom train this dataset using the provided code snippet in a notebook.

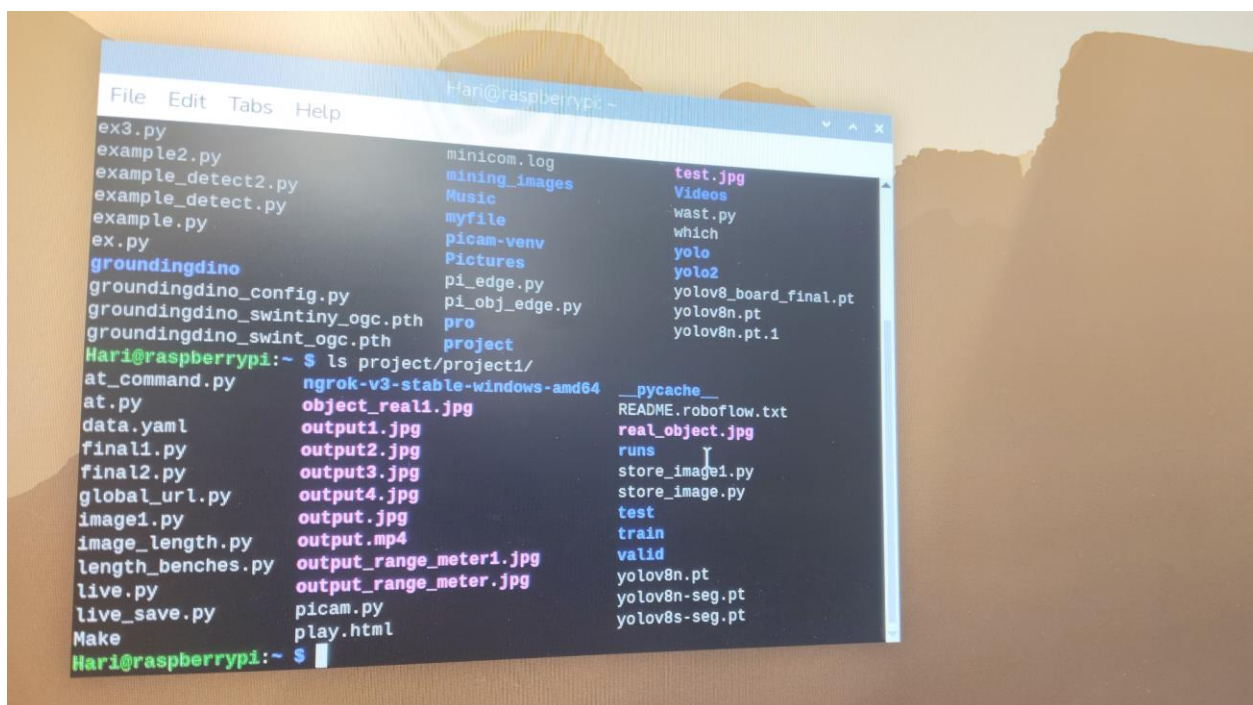
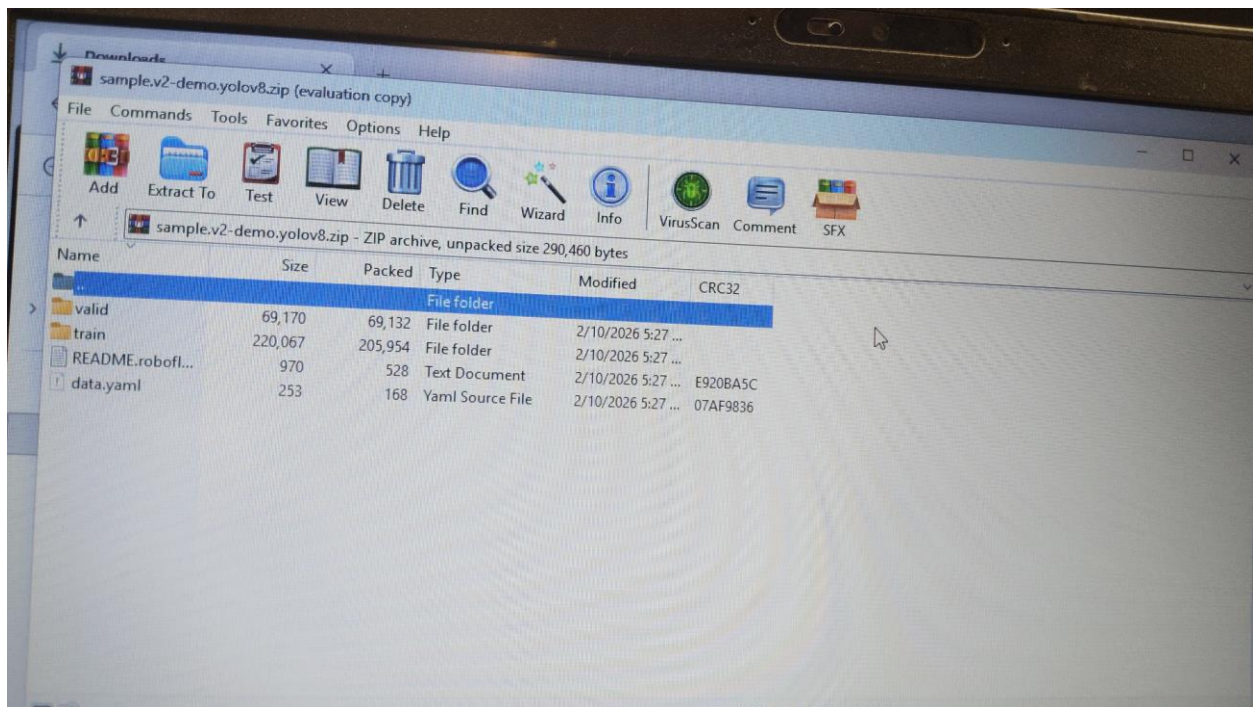
Cancel Continue

4 Total Images

[View All Images](#)



Dataset Split



We can see runs, test, train, valid, [yolov8n.pt](#) files once u download that file

This works perfectly for your kind of projects (camera + YOLO + Raspberry Pi / edge device).

✓STEP 1: Create a Roboflow Account

1. Open: <https://roboflow.com>
2. Click **Sign Up**
3. Login using Google / Email

After login → You'll see **Dashboard**

✓STEP 2: Create a New Project

1. Click **+Create New Project**
2. Fill details:

Field	Value
Project Name	road-detection (example)
Project Type	Object Detection ✓
Annotation Group	(Leave default)
License	Private
Images	Upload later

3.
Click **Create Project**
-

✓STEP 3: Collect Your Images

Before uploading, prepare your images.

☐ Image Rules (Important)

- ✓ Take from **same camera** you'll use later
- ✓ Different angles
- ✓ Different lighting (day/night)
- ✓ Different distances
- ✓ At least:

- Minimum: **100 images**
- Good: **300+ images**

Formats:

☐ .jpg .png

☐ Example:

☐ road1.jpg

road2.jpg

road3.jpg

...

☐

✓STEP 4: Upload Images to Roboflow

1. Open your project
2. Click **Upload**
3. Drag & Drop your images
4. Click **Upload**

Wait until finished.

✓STEP 5: Create Classes (Labels)

Now you tell YOLO **what to detect**.

Example classes:

□ road
person
car
tree
helmet

□ How to Create:

1. Go to **Annotate**
2. Open one image
3. On right side → Click + **Add Class**
4. Type name:

□ road

5. □
Press Enter

Repeat for all classes you want.

✓STEP 6: Annotate (Draw Bounding Boxes)

This is the MOST IMPORTANT step □

You must draw boxes around objects.

How:

1. Select class (example: road)
2. Drag mouse around road
3. Draw box
4. Release

Do for:

- ✓ Every object
- ✓ Every image

Example:

Image	Boxes
-------	-------

img1	road, car
------	-----------

img2	road
------	------

img3	person, bike
------	--------------

☐ Shortcut Tips

Key	Use
-----	-----

B	Draw box
---	----------

D	Delete
---	--------

Space	Move image
-------	------------

→	Next image
---	------------

✓STEP 7: Review Annotations

After labeling:

1. Click **Review**
2. Check:
 - Boxes correct?
 - No missing objects?
 - Wrong class?

Fix mistakes now !

Bad labels = Bad model.

✓STEP 8: Generate Dataset Version

Now Roboflow will prepare dataset for training.

1. Click **Generate**
2. Settings:

Recommended:

Option	Value
Resize	640x640
Augmentation	ON
Flip	ON
Rotate	$\pm 15^\circ$
Blur	ON
Brightness	ON

3.
Click **Generate**

Wait 2–5 min.

This creates **Version 1**.

✓STEP 9: Download Dataset in YOLO Format

After generation:

1. Open Version
2. Click **Download Dataset**
3. Select:

☐ YOLOv8 PyTorch ✓

4. Download ZIP

Unzip:

You'll get:

```
☐ dataset/  
|— train/  
|— valid/  
|— test/  
|— data.yaml  
☐
```

✓STEP 10: Train YOLO Model (On PC / Laptop)

Now training.

Install YOLO:

```
☐ pip install ultralytics
```

☐ **Train:**

Go to dataset folder:

```
☐ yolo detect train data=data.yaml model=yolov8n.pt epochs=50 imgsz=640
```

☐ Explanation:

Option	Meaning
--------	---------

yolov8n.pt	Small fast model
------------	------------------

epochs=50	Training rounds
-----------	-----------------

imgsz=640	Image size
-----------	------------

✓STEP 11: Get Trained Model

After training finishes:

File location:

❑ `runs/detect/train/weights/best.pt`

❑ This is your trained model ✓

Copy it to Raspberry Pi / Project folder.

✓STEP 12: Use Model in Your Python Code

Example:

❑ `from ultralytics import YOLO`

`model = YOLO("best.pt")`

`results = model("test.jpg", show=True)`

❑ Or in camera stream (like your previous projects).

❑ IMPORTANT TIPS (From Experience)

1❑ More Images = Better Accuracy

Try:

❑ 300+ images

❑ 2❑ Always Label Clearly

✗ Bad:

- Half object
- Cropped box

✓ Good:

- Full object
- Tight box

3❑ Use Same Camera

Train with Pi Camera → Use Pi Camera

Don't mix phone + CCTV + webcam.

❑ Recommended Setup for You (Your Projects)

For your road / object projects:

Step	Value
Images	300
Classes	road, person, vehicle

Model yolov8n

Epochs 50–80

Size 640

☐ **FINAL FLOW (Easy Memory)**

☐ Collect Images

↓

Upload to Roboflow

↓

Create Classes

↓

Draw Boxes

↓

Generate Dataset

↓

Download YOLOv8

↓

Train

↓

Get best.pt

↓

Use in Project

☐

```
cat project/project1/final2.py
```

```
cat final3.py
```

```
#!/usr/bin/env python3
```

```
from flask import Flask, Response
```

```
from picamera2 import Picamera2
```

```

import cv2
from ultralytics import YOLO
import subprocess
import threading
import time
import math
import requests
from datetime import datetime

# ----- APP -----
app = Flask(__name__)

# ----- SETTINGS -----

SIM_INTERFACE = "usb0"

CAPTURE_INTERVAL = 2
VM_API_URL = "http://104.154.141.198:3001/upload"
RTMP_URL = "rtmp://104.154.141.198/live/stream1"

# New folder every boot
SESSION_ID = datetime.now().strftime("%Y%m%d_%H%M%S")

print("SESSION:", SESSION_ID)

# ----- GLOBAL FRAME -----

latest_clean = None
frame_lock = threading.Lock()

# ----- SIM SETUP -----

def has_sim_ip():
    return "inet " in subprocess.run(
        ["ip", "addr", "show", SIM_INTERFACE],
        capture_output=True, text=True
    ).stdout

def bring_up_sim():
    subprocess.run(["sudo", "qmi-network", SIM_INTERFACE, "start"])
    time.sleep(6)

```



```

def add_route():
    subprocess.run(["sudo","ip","route","del","default"],
                    stderr=subprocess.DEVNULL)

    subprocess.run(["sudo","ip","route","add","default","dev",SIM_INTERFACE])

def ping_test():
    return "0% packet loss" in subprocess.run(
        ["ping","-c","2","8.8.8.8"],
        capture_output=True,text=True
    ).stdout

print("Checking SIM...")

if not has_sim_ip():
    bring_up_sim()

if has_sim_ip():
    add_route()

print("SIM OK" if ping_test() else "SIM FAIL")

# ----- CAMERA -----

print("Starting camera...")

picam2 = Picamera2()

config = picam2.create_video_configuration(
    main={"size":(640,480),"format":"RGB888"},
    controls={"FrameRate":30}
)

picam2.configure(config)
picam2.start()

time.sleep(2)

print("Camera ready")

```

```

# ----- YOLO -----

print("Loading YOLO...")

model = YOLO("runs/segment/train4/weights/best.pt")

print("YOLO loaded")


# ----- FFMPEG -----

print("Starting FFmpeg...")

ffmpeg = subprocess.Popen([

    "ffmpeg", "-y",

    "-f", "rawvideo",
    "-pix_fmt", "bgr24",
    "-s", "640x480",
    "-r", "25",
    "-i", "-",

    "-c:v", "libx264",
    "-preset", "ultrafast",
    "-tune", "zerolatency",

    "-b:v", "1200k",
    "-maxrate", "1200k",
    "-bufsize", "2400k",

    "-f", "flv",
    RTMP_URL

], stdin=subprocess.PIPE)


# ----- IMAGE UPLOAD THREAD -----

def upload_loop():

    global latest_clean

```

```

last = 0

while True:

    if time.time() - last < CAPTURE_INTERVAL:
        time.sleep(0.1)
        continue

    with frame_lock:

        if latest_clean is None:
            continue

        frame = latest_clean.copy()

    _, jpg = cv2.imencode(
        ".jpg",
        frame,
        [int(cv2.IMWRITE_JPEG_QUALITY), 90]
    )

    files = {
        "image": ("img.jpg", jpg.tobytes(), "image/jpeg")
    }

    data = {
        "session_id": SESSION_ID
    }

    try:

        r = requests.post(
            VM_API_URL,
            files=files,
            data=data,
            timeout=5
        )

        if r.status_code == 200:
            print("[✓] Uploaded")

    except:
        print("[!] Upload failed")

```

```
last = time.time()
```

```
# ----- FRAME GENERATOR -----
```

```
def gen_frames():
```

```
    global latest_clean
```

```
    cx = 320
```

```
    cy = 240
```

```
    while True:
```

```
        # Capture
```

```
        frame = picam2.capture_array()
```

```
        clean = frame.copy()
```

```
        # YOLO detect
```

```
        results = model(frame)[0]
```

```
        for box, cls in zip(results.bboxes.xyxy,  
                             results.bboxes.cls):
```

```
            x1,y1,x2,y2 = box.cpu().numpy()
```

```
            w = x2-x1
```

```
            h = y2-y1
```

```
            mx = x1 + w/2
```

```
            my = y1 + h/2
```

```
            angle = math.degrees(  
                math.atan2(my-cy,mx-cx)  
            )
```

```
            label = results.names[int(cls)]
```

```
            text = f'{label} W:{w:.1f} H:{h:.1f} A:{angle:.1f}'
```

```
# Draw rectangle
cv2.rectangle(
    frame,
    (int(x1),int(y1)),
    (int(x2),int(y2)),
    (0,0,255),2
)
```

```
# Text position near box
tx = int(x1)
ty = int(y1) - 8
```

```
if ty < 15:
    ty = int(y1) + 15
```

```
# Draw RED text (no background)
cv2.putText(
    frame,
    text,
    (tx, ty),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.5,
    (0,0,255), # RED
    1,
    cv2.LINE_AA
)
```

```
# Save clean frame for upload
with frame_lock:
    latest_clean = clean
```

```
# Send to RTMP
try:
```

```
    ffmpeg.stdin.write(
        cv2.cvtColor(
            frame,
            cv2.COLOR_RGB2BGR
```

```

        ).tobytes()
    )

except:
    print("[!] FFmpeg issue")

# Send to browser
_, jpg = cv2.imencode(
    ".jpg",
    frame,
    [int(cv2.IMWRITE_JPEG_QUALITY), 80]
)

yield (

    b"--frame\r\n"
    b"Content-Type: image/jpeg\r\n\r\n" +
    jpg.tobytes() + b"\r\n"

)

# ----- FLASK -----

@app.route("/")
def index():
    return "<img src='/stream'>"

@app.route("/stream")
def stream():
    return Response(
        gen_frames(),
        mimetype="multipart/x-mixed-replace; boundary=frame"
    )

# ----- START -----

if __name__ == "__main__":

    print("Starting upload thread...")

```

```

threading.Thread(
    target=upload_loop,
    daemon=True
).start()

print("Starting server...")

app.run(
    host="0.0.0.0",
    port=5001,
    threaded=True
)

```

how to check all services running in rpi

`systemctl list-units -- type=service --state=running`

Live video stream code running in this server

`sudo systemctl status ec200-stream.service`

In this service running final code of live stream that is final3.py

`/ $ sudo nano /etc/systemd/system/ec200-stream.service`

[Unit]

Description=EC200 Cellular Live Stream

After=network.target

Wants=network.target

[Service]

Type=simple

User=Hari

WorkingDirectory=/home/Hari/project/project1

ExecStart=/home/Hari/picam-venv/bin/python3 /home/Hari/project/project1/final3.py

Restart=always

RestartSec=10

Environment=PYTHONUNBUFFERED=1

[Install]

WantedBy=multi-user.target

In this run at commands to enable the net from ec200

```
sudo cat /usr/local/bin/ec200-init.sh
```

```
#!/bin/bash
```

```
LOG=/var/log/ec200-init.log
```

```
echo "==== EC200 init started: $(date) =====>> $LOG
```

```
# Wait for USB devices to appear
```

```
for i in {1..20}; do
```

```
    PORT=$(ls /dev/ttyUSB* 2>/dev/null | head -n 1)
```

```
    if [ -n "$PORT" ]; then
```

```
        break
```

```
    fi
```

```
    echo "Waiting for ttyUSB... ($i)">> $LOG
```

```
    sleep 1
```

```
done
```

```
if [ -z "$PORT" ]; then
```

```
    echo "✗No ttyUSB port found">> $LOG
```

```
    exit 1
```

```
fi
```

```
echo "✓Using port: $PORT">> $LOG
```

```
# Configure serial port
```

```
stty -F "$PORT" 115200 raw -echo
```

```
send_at () {
```

```
    echo -e "$1\r" > "$PORT"
```

```
    sleep 1
```

```
    timeout 2 cat "$PORT">> $LOG
```

```
}
```

```
send_at "AT"
```

```
send_at "AT+CSQ"
```

```
send_at "AT+CPIN?"
```

```
send_at "AT+QNETDEVCTL=1,1,1"
```

```
echo "==== EC200 init finished: $(date) =====>> $LOG
```

continuous running above .sh file

-> `sudo nano /etc/systemd/system/ec200-init.service`

That is [ec200-init.sh](#)

~ \$ `sudo cat /etc/systemd/system/ec200-init.service`

[Unit]

Description=EC200 Modem Init AT Commands

After=network.target usb.target

Wants=network.target

[Service]

Type=oneshot

ExecStart=/usr/local/bin/ec200-init.sh

RemainAfterExit=yes

[Install]

WantedBy=multi-user.target

Tailscale Funnel Setup & Auto-Recovery (Port 443 Fix)

1 ☐ Check Current Tailscale Serve Status

```
tailscale serve status --json
```

This shows the active serve/funnel configuration.

2 ☐ Start Funnel on Port 5001

```
sudo tailscale funnel 5001
```

☐ If You See This Error:

sending serve config: updating config: listener already exists for port 443

It means **port 443 is already occupied** by a previous funnel/serve process.

3 ☐ Free Port 443

Disable the old HTTPS listener:

```
sudo tailscale serve --https=443 off
```

4 ☐ Restart Funnel

```
sudo tailscale funnel 5001
```

Expected Output:

Available on the internet:

<https://myservice.taileb4769.ts.net/>

|-- proxy http://127.0.0.1:5001

Press Ctrl+C to exit.

Now your global URL works:

<https://myservice.taileb4769.ts.net/stream>

5 ☐ Run Serve in Background Mode

```
sudo tailscale serve --bg 5001
```

This ensures the local service keeps running.

☐ Automatic Funnel Restart After Reboot

Since sometimes **port 443 remains busy after reboot**, we created a script to:

- Check port status
 - Disable old listener if needed
 - Restart funnel automatically
-

☐ Script Location

~/start_funnel.sh

Example `cat ~/start_funnel.sh`

```
#!/bin/bash
```

```
echo "Stopping old Tailscale configs..."
```

```
# Reset everything
```

```
sudo tailscale serve reset
```

```
sudo tailscale funnel reset
```

```
sleep 2
```

```
# Kill anything using 443 (Tailscale HTTPS)
sudo fuser -k 443/tcp || true
```

```
sleep 2
```

```
# Kill anything using 5001 (Flask safety)
sudo fuser -k 5001/tcp || true
```

```
sleep 2
```

```
echo "Starting Funnel..."
```

```
# Start funnel
sudo tailscale funnel 5001
```

☐ **Systemd Service (Runs on Boot)**

Service File Location:

```
sudo cat etc/systemd/system/tailscale-funnel.service
```

[Unit]

Description=Tailscale Funnel Auto Start

After=network-online.target tailscaled.service

Wants=network-online.target

[Service]

Type=simple

User=Hari

ExecStart=/home/Hari/start_funnel.sh

Restart=always

RestartSec=15

[Install]

WantedBy=multi-user.target

Enable Service

```
sudo systemctl daemon-reload
sudo systemctl enable tailscale-funnel.service
sudo systemctl start tailscale-funnel.service
```

Check status:

```
sudo systemctl status tailscale-funnel.service
```

✓ Final Result

After every reboot:

Example Public URL:

`https://myservice.taileb4769.ts.net/stream`

1 ☐ Check Your Tailscale HTTPS Domain Name

Run:

```
tailscale status
```

You will see something like:

```
100.x.x.x raspberrypi hari@ linux active
```

`tailscale ip -4` → it shows an IP

It will show:

```
https://myservice.taileb4769.ts.net
|-- proxy http://127.0.0.1:5001
```

☐ That `myservice.taileb4769.ts.net` is your **device URL name**.

✓2 ☐ Check From Browser

Open:

`https://myservice.taileb4769.ts.net`

If your Flask stream is running on port 5001, then check:

`https://myservice.taileb4769.ts.net/stream`

If it loads → ✓Working

If not → check funnel status.

✓3 ☐ Confirm Funnel is Active

Run:

`sudo tailscale funnel status`

Or:

`tailscale serve status`

If funnel is active, it will show:

Funnel: on

HTTPS: 443

✓4 ☐ If You Want to Know Where That Name Comes From

The name:

`myservice.taileb4769.ts.net`

Is generated automatically by **Tailscale** based on:

- Your tailnet name

- Your device hostname
- Your account domain

You can also see it inside:

☐ Tailscale Admin Console

<https://login.tailscale.com/admin/machines>

Under your device → it shows the full DNS name.
